

Agentic AI & LangGraph

Mukammal Study Notes — Roman Urdu

Agents • ReAct • Multi-Agent Systems • Deep Agents
LangGraph Architecture • Tool Engineering & Pydantic •
Memory • Human-in-the-Loop
+ Extra Agentic AI Concepts & Complete Interview
Question Bank

Compiled & expanded from class notes — duplicate content removed,
missing agentic AI concepts added for complete interview coverage.

Agentic AI & LangGraph — Mukammal Notes (Roman Urdu)

Interview Preparation Guide

Ye notes aapke original class notes se banaye gaye hain — repeated cheezein hata kar, aur jo important agentic AI concepts notes mein miss thay wo add kar ke (taake interview mein koi gap na ho).

SECTION 1: Foundations — Assistant se Agentic AI tak

1.1 Simple AI Assistant

Ek simple AI Assistant sirf **Input → LLM → Output** ka flow follow karta hai. Jawab sirf model ki training se aata hai, bahar se kuch retrieve nahi hota.

Example: "Tesla kya hai?" → "Tesla ek EV company hai" (ye jawab model ne apni training se diya, kisi database se nahi).

1.2 RAG-based AI Assistant

RAG (Retrieval-Augmented Generation) assistant pehle ek **retrieval step** add karta hai — kisi external data source (database/documents) se relevant context nikalta hai, phir us context + query ko LLM mein daal kar zyada grounded (up-to-date, factual) jawab banata hai.

User Query → Data Source → Relevant Context Retrieve → (Query + Context + Instructions) → LLM → Output

1.3 AI Agent Kya Hai

AI Agent = Autonomous/Intelligent System. Ye aisa system hai jo khud decide kar sakta hai ke kya action lena hai, tools use kar sakta hai, aur environment se feedback le kar goal complete kar sakta hai. **LLM agent ka "dimagh" (brain) hota hai** — saari reasoning/thinking LLM se aati hai.

Flow: User Goal → Think (LLM) → Plan (agar zaroori ho) → Action (tool call) → Observe result → Loop repeat jab tak goal complete na ho.

Class Example — Tesla Investment Report: - Current financial info chahiye → Web Search tool - Historical stock data → Finance API - Calculations → Python tool - Annual report → Document search - "Kya enough info hai?" → Nahi → dobara search; Haan → Report generate karo

1.4 Agentic AI Kya Hai

Agentic AI ek umbrella term hai — har wo AI application jo **agents** ke concept par bani ho. Ye do types mein divide hoti hai: - **Single/Mono-Agentic System** — ek agent hi pura task karta hai - **Multi-Agentic System** — multiple specialized agents mil kar kaam karte hain

SECTION 2: Agentic Flow — Think → Act → Observe → Adapt

2.1 Deterministic Flow vs Dynamic Flow

Deterministic Flow	Dynamic Flow
Developer manually routing logic likhta hai ("agar X to node Y")	LLM khud runtime par decide karta hai kaunsa tool/node call karna hai
Supervisor node fixed nodes ko dispatch karta hai	LLM dynamically decision leta hai — ye tool-calling ki base hai
Predictable, debug karna asaan, lekin har branch developer ko sochni parti hai	Flexible, general-purpose, lekin control karna mushkil

Instructor ki baat: Dono hi "agents" hain — bas farq itna hai ke decision kaun le raha hai (developer ka likha logic ya khud LLM).

2.2 ReAct Loop (Reasoning + Action)

ReAct ka matlab hai **Reasoning + Action** ka combination. Agent ye loop follow karta hai:

```
Input → Think/Reasoning → Planning (agar chahiye) → Action (tool call) → Observe result → Adapt → Output
```

Ye loop tab tak chalta hai jab tak LLM khud decide na kare ke goal complete ho gaya hai.

SECTION 3: Multi-Agent Systems & Deep Agents

3.1 Multi-Agent System

Ek Supervisor Agent input leta hai, reason karta hai, aur sub-tasks ko specialized worker agents ko delegate kar deta hai.

Tesla Example: - **Research Agent** → products, market, competitors - **Financial Agent** → revenue, profit, cash flow - **Risk Agent** → regulatory/supply-chain risks - **Market Agent** → EV market, market share - **Writer Agent** → sab combine kar ke final report banata hai

Fayda: Ek agent sab kuch karne ke bajaye, har agent apni specialty mein kaam karta hai — modularity aur manageability better hoti hai.

3.2 Deep Agentic System

Deep Agent un **bade, long-running, complex problems** ke liye banaya jata hai jinko extensive planning, task decomposition, multiple tools/sub-agents chahiye hote hain.

Flow: Complex Goal → Extensive Planning → Task Decomposition → Multiple Tools → Sub-Agents → Repeat → Final Deliverable

Tesla Example (Deep Agent): Pehle TODO plan banega (download reports → extract data → research competitors → analyze market → calculate ratios → charts → risks → compare → verify → write report), phir ek working file structure banti hai:

```
/workspace
/tesla_financials.csv
/competitor_analysis.md
/market_research.md
/risk_analysis.md
/charts/
/sources.md
/final_report.docx
```

Context management: Agar research bohat zyada ho jaye to → Summarize karo / Files mein store karo → sirf relevant context rakho → kaam continue.

3.3 Shallow Agent vs Deep Agent

- **Shallow Agent** — single/mono-agentic, simple task
- **Deep Agent** — complex, long-running problem, multi-step planning, possibly multiple sub-agents underneath

SECTION 4: Agent Frameworks Landscape

#	Framework	Backed By
1	Pure Python	From scratch
2	★ LangGraph (course ka winner)	LangChain — sabse zyada popular
3	OpenAI Agent SDK	OpenAI
4	Google ADK	Google
5	Autogen	Microsoft
6	CrewAI	—
7	SmolAgent	HuggingFace
8	n8n	No-code framework

LangGraph kyun chosen hai: Isme graph-based structure milta hai (nodes, edges, state) jo hand-written control flow se better organize hota hai. Course ke saare hands-on examples (ReAct agent, supervisor/router, memory, HITL) LangGraph mein banaye gaye hain.

SECTION 5: Core Agentic Engineering Concepts

Agentic system ki capabilities (10): 1. Planning/Dynamic planning 2. Execution 3. Task decomposition 4. Reflection 5. Self-correction/healing/critic 6. Reasoning 7. Parallelization 8. Orchestration 9. Human-in-the-loop 10. Feedback-aware optimization

Ye sab Think→Act→Observe loop aur multi-agent orchestration se related hain.

Production agents banane ke liye required skills (14): Pydantic/enum/type hints, JSON schema, Async Python, Structured Output, Tool engineering, Memory management, State management, Prompting, Context engineering, MCP/A2A Protocol, Evaluation/testing, Observability, Guardrails/Security, Caching.

SECTION 6: LangGraph Fundamentals

6.1 LangGraph vs LangChain vs LangSmith

- **LangChain** — LLM applications banane ka framework (chains, prompts, integrations)
- **LangGraph** — graph-based orchestration layer, jisme aap state, nodes, edges define kar ke complex agent workflows banate hain
- **LangSmith** — observability/monitoring tool, jisse aap traces dekh sakte hain ke agent ne kaunse steps liye

6.2 Core Building Blocks

- **State** — shared data object jo nodes ke darmiyan pass hota hai (e.g. question, route, context, draft_answer)
- **Node** — function (ya LLM call) jo state read kar ke usme updates return karta hai
- **Edge** — nodes ko connect karta hai; **fixed edge** ya **conditional edge** (jo state ke basis par dynamically route kare)
- **Checkpoint** (e.g. InMemorySaver) — graph state ka snapshot save karta hai, isse resumability aur human-in-the-loop interrupts possible hote hain

6.3 LangGraph Topic Roadmap

Introduction → Architecture & Tools → State Management → StateGraph vs Graph vs Functional API → Send API/Map-Reduce → Command API → Persistence/Checkpoint → Memory → Human-in-the-loop → Streaming → Multi-agent → Caching → Fault Tolerance → Async LangGraph

SECTION 7: ReAct Agent — LangGraph mein Kaise Banate Hain

7.1 Graph Structure

```
__start__
  ↓
llm_node (LLM tools ke saath bound — llm_with_tool)
  ↓
Conditional edge: "Kya tool call zaroori hai?"
  ↓ (haan)                ↓ (nahi)
tools node (function execute)  __end__
  ↓
wapis llm_node (loop)
```

Key question jo graph solve karta hai: "LLM kaise decide karega ke kaunsa tool call karna hai?"

Example tool set: [Add, Sub, Mul, Div, Pow]

7.2 Worked Example: "2+2 kya hai, aur uska output 10 se multiply karo?"

1. Prompt + reasoning → llm_with_tool ko bheja
2. LLM decide karta hai: pehle **Add** tool call karo (a=2, b=2) → result = 4
3. LLM us output ko le kar decide karta hai: **Mul** tool call karo (a=4, b=10) → result = 40
4. Ab koi tool zaroori nahi → `__end__` → final answer: **40**

Ye kya show karta hai: Ye dynamic flow ka live example hai — LLM khud, har step par, decide kar raha hai kaunsa tool call karna hai aur kya arguments dene hain, developer ne "pehle Add phir Mul" hardcode nahi kiya.

SECTION 8: Deterministic Orchestration — Supervisor/Router Pattern

Isme ek Supervisor/Router node state inspect kar ke manually fixed nodes ko route karta hai (LLM ki jagah developer logic decide karta hai).

Example — RAG/LLM/Web Router: Router state ke fields padhta hai: `route`, `route_reasoning`, `attempt`, `used_routes`, `context`, `draft_answer`. `route` ki value (rag/llm/web) ko dictionary-key ki tarah match kar ke sahi node par dispatch karta hai.

```
state.route == "rag" → RAG node
state.route == "llm" → LLM node
state.route == "web" → Web-search node
```

Instructor ka summary: Deterministic agent mein LLM bhi process ka part hota hai lekin routing developer likhta hai. Dynamic (ReAct-style) agent mein tool-calling capability LLM ko di jati hai, koi manual if/else nahi hota.

SECTION 9: Tool Engineering & Pydantic Schemas

Tool engineering ka matlab hai function ke inputs ko itna precisely describe karna ke LLM ko exactly pata ho kya pass karna hai. Warna galat data type (string ki jagah float) execution break kar sakta hai.

args vs *kwargs

- `*args` → positional values ka tuple, koi bhi number of arguments accept karta hai
- `**kwargs` → keyword arguments ka dictionary, e.g. `{name: "Sunny", age: 30, city: "Bangalore"}`
- Calculator jaisa tool banate waqt numeric argument ko explicitly `float` type dena chahiye, warna LLM confuse ho sakta hai.

Pydantic Model Basics

```
class User(BaseModel):
    name: str
    country: str = "India"          # default value
    middle_name: str | None = None  # optional
    nickname: str | None = None    # optional

User(name="Sunny", country="USA", middle_name="Kumar", nickname="Sun")
# -> works fine

User(name="Sunny", middle_name="KUMAR")
# -> bhi works: country default "India" ban jayega, nickname None rahega
```

- Field pass na ho aur default set ho to Pydantic khud default fill kar deta hai
- `None` genuinely optional fields ke liye use hota hai
- Yahi schema discipline tool arguments par bhi apply hoti hai — taake LLM ka tool call validated aur type-safe ho

SECTION 10: Memory in LangGraph

10.1 Memory Kyun Zaroori Hai

Memory ke bagair har turn stateless hota hai — agent pichle turns bhool jata hai. LangGraph memory conversation history (aur purane turns ka compressed summary) maintain karti hai, taake prompt size unboundedly grow na ho.

10.2 Summary Memory + Sliding Window

Memory class ke methods: - `create_summary` — purane turns ka initial summary banana - `compress_summary` — naye purane messages ko running summary mein fold karna - `add_user_message` / `add_assistant_message` — naye turns memory mein append karna - `get_recent_conversation` — sabse recent raw messages return karta hai, ya "no recent conversation"

Compression rule: "Compress sirf ek complete user-assistant turn ke baad" — yani summarization mid-turn nahi, pura exchange complete hone ke baad trigger hoti hai.

```
New user input → Full history + summary memory mein append
→ Prompt banao = system + purana summary + recent raw messages
→ Model ko bhejo → Response generate ho
→ History mein append karo → turn complete hone ke baad summary recompute karo
```

Sliding-window detail: Purana summary earlier messages ka context capture karta hai (e.g. pehle 4 messages ka summary), jabke latest messages (e.g. last 6) raw form mein rakhe jate hain. Jaise conversation badhti hai, window aage slide karti hai — purane raw messages summary mein fold ho jate hain, naye raw rahte hain. Isse prompt size bounded rehta hai lekin recent detail preserve hoti hai.

Sirf raw kyun nahi rakhte? Agar hum direct recent messages hi use karein to summary ki zaroorat hi nahi — summarization specifically tab chahiye jab history itni badi ho jaye ke poori pass nahi ki ja sakti.

10.3 Query Rewriting (Conversational Retrieval ke liye)

`rewrite_query` method follow-up question ko ek standalone query mein convert karta hai, taake retrieval unresolved pronouns/context par depend na kare.

```
get_recent_conversation() → recent memory (ya "no recent conversation")
→ Rewrite-prompt banao (recent memory + follow-up question)
→ Model ko bhejo → standalone query generate hoti hai
→ retrieve_query(standalone query) → relevant context fetch
```

Ye standard conversational-RAG technique hai — context-dependent follow-ups ("uske baare mein aur batao?") ko fully self-contained query mein resolve karna, retriever ke pass conversation history nahi hoti isliye.

SECTION 11: Human-in-the-Loop (HITL)

HITL LangGraph workflow ko kisi sensitive step par pause kar deta hai aur human decision ka wait karta hai — high-stakes actions (jaise financial refunds) ke liye critical, jahan full autonomy risky hai.

Worked Example — Refund Approval:

```
start → analyze (agent request inspect karta hai)
↓
human_approval ← graph yahan pause/interrupt hota hai
↓
approve_router(): "execute" → execute → end
approve_router(): "reject" → end (process stop)
```

Demo input: "Customer101 ko damaged product mila hai, refund kar do." Graph `human_approval` tak chalta hai, interrupt hota hai, aur review ke liye details print karta hai (action=refund, customer ID, amount, region) + message: "Graph paused for human approval."

Implementation Building Blocks

- **Checkpoint (InMemorySaver)** — graph state ka snapshot save karta hai
- Graph ko checkpoint ke saath compile karna
- **Config** — `thread_id` include karta hai, jo specific run identify karta hai
- **approve_router** — conditional-edge function jo human ka decision padh kar execute/end route karta hai

- **Resuming** — human ka yes/no input le kar `Command` ke zariye same thread config ke saath graph mein wapis bhejna — graph exactly wahi se continue hota hai jahan pause hua tha

SECTION 12: Guardrails & Agent Control Parameters

Agent loop ko runaway hone se rokne ke liye (infinite loops, cost blowup): - **Maximum iteration limit** — Think→Act→Observe cycles ki max count - **Timeout** — wall-clock time cap - **Stop condition** — explicit condition jo loop ko jaldi khatam kar de - **Token/cost budget** — per-run spend cap

SECTION 13: EXTRA CONCEPTS (Notes mein Nahi Thay — Interview ke liye Zaroori)

Ye section original class notes mein nahi tha, lekin Agentic AI/LangGraph interviews mein ye poocha jana common hai.

13.1 Agentic Design Patterns (Andrew Ng ka Framework)

Agentic AI ki industry mein 4 mashhoor design patterns hain: 1. **Reflection** — agent apna khud ka output review/critique karta hai aur improve karta hai (self-correction) 2. **Tool Use** — agent external tools/APIs call karta hai (calculator, search, database) 3. **Planning** — agent complex task ko chhote steps mein break karta hai aur ek plan banata hai 4. **Multi-Agent Collaboration** — multiple agents mil kar specialized roles ke sath kaam karte hain

13.2 Agentic RAG

Normal RAG mein retrieval ek fixed, single step hota hai. **Agentic RAG** mein LLM khud decide karta hai: - Kya retrieval zaroori hai ya nahi - Kaunse data source se retrieve karna hai (multiple sources ho sakte hain) - Agar retrieved context insufficient ho to dobara query rewrite kar ke retry kare Ye RAG ko static pipeline se dynamic, self-correcting loop banata hai.

13.3 Embeddings & Vector Databases

- **Embedding** — text/image ko numeric vector mein convert karna, taake semantic similarity measure ho sake
- **Vector Database** (Pinecone, Chroma, Weaviate, FAISS) — embeddings store karta hai aur similarity search (top-k nearest neighbors) enable karta hai
- RAG/Agentic RAG ka core infrastructure yahi hota hai

13.4 MCP (Model Context Protocol) & A2A Protocol

- **MCP** — Anthropic ka open protocol jo LLM ko external tools/data sources se standardized tareeke se connect karta hai (jaise USB-C port sab devices ke liye). Isse har tool ke liye custom integration nahi likhni parti.
- **A2A (Agent-to-Agent) Protocol** — Google ka protocol jo different agents (chahe different frameworks/vendors ke ho) ko aapas mein communicate karne deta hai — multi-agent interoperability ke liye.

13.5 LangGraph Advanced Mechanisms

- **Send API / Map-Reduce** — ek node se dynamically multiple parallel nodes ko fan-out kar sakte hain (e.g. ek list ke har item par alag processing), phir results ko fan-in/reduce kar sakte hain

- **Command API** — node se return hokar state update + next node dono control kiye ja sakte hain ek hi object mein
- **Subgraphs** — graph ke andar chhote independent graphs embed kar sakte hain (modularity, reusability)
- **Streaming** — LangGraph tokens/state updates ko real-time stream kar sakta hai, taake UI turant response dikha sake
- **Caching** — repeated/expensive node calls (LLM calls, tool calls) ka result cache kar ke cost aur latency kam karna
- **Fault Tolerance** — agar koi node fail ho to graph retry ya graceful recovery kar sake, checkpointing ki madad se

13.6 Agent Evaluation & Observability

- **Evaluation** — agent ke outputs ko accuracy, task-completion rate, aur correctness ke against test karna (e.g. LangSmith evals, custom test cases)
- **Observability** — traces dekhna ke agent ne kaunse steps liye, kaunse tools call kiye, kitna time/tokens lage — debugging aur production monitoring ke liye zaroori (LangSmith is tool ka example hai)

13.7 Context Engineering

Sirf prompt engineering kaafi nahi — **context engineering** ka matlab hai ke LLM ko right time par right information (tools, memory, retrieved docs, past state) dena, taake context window overflow na ho aur relevant info hamesha available rahe.

13.8 Historical Context — AutoGPT / BabyAGI

Early autonomous agent experiments (2023) — AutoGPT aur BabyAGI ne pehli baar demonstrate kiya ke LLM khud task decompose kar ke, tools use kar ke, loop mein kaam kar sakta hai bina human intervention ke. Ye modern frameworks (LangGraph, deep agents) ka conceptual foundation hain, lekin unreliable/looping issues ki wajah se production mein directly use nahi hote.

SECTION 14: Interview Question Bank (Deduped & Complete)

A. Conceptual — Agentic AI Fundamentals

Q1. Simple AI Assistant aur RAG Assistant mein farq? Simple Assistant sirf model ki trained knowledge se jawab deta hai (Input→LLM→Output). RAG Assistant pehle external data source se relevant context retrieve karta hai, phir Query+Context+Instructions LLM ko deta hai — zyada grounded, up-to-date jawab.

Q2. AI Agent, simple assistant se kaise different hai? Assistant sirf text response generate karta hai. AI Agent autonomous system hai jo tools use kar ke actions le sakta hai, environment se feedback le kar decide kar sakta hai, plan kar sakta hai, aur goal complete hone tak loop kar sakta hai.

Q3. Agent ke andar LLM ka role kya hai? LLM agent ka "brain" hai — saari reasoning, planning, decision-making (kaunsa tool call karna, kab rukna) LLM se aati hai. Tools sirf functions hain, LLM decide karta hai kab/kaise use karna hai.

Q4. ReAct pattern explain karein. ReAct = Reasoning + Action. LLM next step sochta hai, action leta hai (tool call), result observe karta hai, aur ye loop repeat hota hai jab tak final answer na mil jaye. Dynamic flow isliye hai kyunke LLM har step runtime par khud decide karta hai.

Q5. Deterministic vs Dynamic flow mein farq? Deterministic mein developer routing manually likhta hai (supervisor explicitly next node decide karta hai). Dynamic mein LLM ko tool-calling capability di jati hai aur wo khud runtime par decide karta hai — koi hardcoded if/else nahi.

Q6. Multi-Agent System kya hai aur kyun use karte hain? Multiple specialized agents ek hi agent ki jagah kaam karte hain. Supervisor Agent input le kar sub-tasks ko worker agents ko delegate karta hai (e.g. Research, Financial, Risk, Writer), phir outputs combine karta hai. Ye modularity aur specialization deta hai vs ek monolithic agent.

Q7. Deep Agent kya hai, kab use karte hain? Bade, long-running, complex problems ke liye — extensive planning, task decomposition, multiple tools/sub-agents chahiye hote hain. Jaise TODO plan banana, step-by-step execute karna, intermediate files workspace mein save karna, aur bade research ko summarize karke context manage karna.

Q8. Shallow vs Deep Agent? Shallow Agent simple task ke liye single/mono-agentic system hai. Deep Agent complex problem ke liye multi-step planning aur possibly multiple sub-agents/tools orchestrate karta hai.

Q9. Agentic system ki core capabilities list karein. Planning, execution, task decomposition, reflection, self-correction, reasoning, parallelization, orchestration, human-in-the-loop, feedback-aware optimization.

Q10. Popular agent frameworks aur unka comparison? Pure Python (scratch), LangGraph (LangChain — most used), OpenAI Agent SDK, Google ADK, Autogen (Microsoft), CrewAI, SmolAgent (HuggingFace), n8n (no-code).

Q11 (Extra). Andrew Ng ke 4 agentic design patterns kya hain? Reflection (self-critique), Tool Use (external tools call karna), Planning (task decomposition), Multi-Agent Collaboration (specialized agents mil kar kaam karte hain).

Q12 (Extra). Agentic RAG normal RAG se kaise different hai? Normal RAG mein retrieval fixed, single step hoti hai. Agentic RAG mein LLM khud decide karta hai ke retrieval chahiye ya nahi, kaunsa source use karna hai, aur agar context insufficient ho to query rewrite kar ke retry karta hai — dynamic aur self-correcting.

B. LangGraph & Implementation

Q13. LangGraph graph ke core building blocks? State (shared data), Node (function/LLM call jo state update karta hai), Edge (fixed ya conditional connection), Checkpointer (state snapshot, resumability aur HITL ke liye).

Q14. Typical ReAct agent ka graph structure describe karein. `__start__` → `llm_node` (tools bound) → conditional edge check karta hai tool call chahiye ya nahi → agar haan, `tools` node execute ho kar wapis `llm_node` loop karta hai → agar nahi, `__end__`.

Q15. "2+2 phir 10 se multiply" worked example walk-through. LLM pehle Add tool call karta hai (a=2,b=2) → result 4. Phir Mul tool call (a=4,b=10) → result 40. Koi aur tool zaroori nahi to graph end ho kar 40 return karta hai — LLM dynamically chained tool calls kar raha hai.

Q16. Router/Supervisor node next node kaise decide karta hai? Shared state ke fields padhta hai (route, route_reasoning, attempt, used_routes, context, draft_answer) aur `route` value (rag/llm/web) ko dictionary-key की तरह match kar ke correspondingly named node par dispatch karta hai.

Q17. Tool arguments LLM ke liye tricky kyun hote hain? Kyunke tool functions `*args` (positional tuple) ya `**kwargs` (named dictionary) accept kar sakte hain jinke types ambiguous ho sakte hain. Explicit schema (Pydantic, e.g. numeric field ko float) ke bagair LLM galat data type pass kar sakta hai, jisse tool call break ho jata hai.

Q18. Tool schema define karte waqt Pydantic kyun use karte hain? Pydantic required vs optional fields, default values, aur strict types declare karne deta hai, jisse tool calls automatically validate hote hain — malformed/mistyped LLM-generated arguments se runtime errors kam hoti hain.

Q19. InMemorySaver jaisa checkpointer LangGraph mein kya purpose serve karta hai? Graph state ka snapshot persist karta hai taake execution pause/resume ho sake — HITL flows ke liye zaroori, jahan graph interrupt ho kar human decision ka wait karta hai aur same `thread_id` config se exactly wahi se continue

karta hai.

Q20. LangGraph ka human-in-the-loop pattern end-to-end kaise kaam karta hai? Graph normally chalta hai (e.g. start→analyze) jab tak `human_approval` node tak nahi pohanchta, jahan interrupt ho kar action review ke liye surface hota hai. Human decision (yes/no) `Command` ke zariye same `thread_id` ke saath wapis bheja jata hai, aur `approve_router` conditional edge execute/reject route karta hai.

Q21. LangGraph-based chatbot mein multi-turn memory kaise implement karenge bina prompt ko unbounded grow kiye? Summary-memory design use karen: recent N raw messages verbatim rakhein, purane messages ko periodically ek running summary mein compress karen (sirf complete turn ke baad). Har naye call ka prompt = system + summary + recent raw messages — total size bounded rehti hai.

Q22. Conversational RAG agent mein query rewriting kyun zaroori hai retrieval se pehle? Query rewriting context-dependent follow-up ("uske baare mein aur batao?") ko fully self-contained standalone query mein convert karta hai, recent conversation history use kar ke. Ye standalone query phir retrieval ke liye use hoti hai, kyunke retriever ke pass khud conversation context nahi hota.

Q23. Production agent ko kaunse guardrail parameters enforce karne chahiye, aur kyun? Maximum iteration limit aur timeout (infinite/runaway loops rokne ke liye), stop conditions (goal complete hote hi loop end), token/cost budget (per-run spend cap) — kyunke autonomous Think→Act→Observe loop warna indefinitely chal sakta ya bohat expensive ho sakta hai.

Q24 (Extra). LangGraph ka Send API / Map-Reduce kya karta hai? Ek node se dynamically multiple parallel nodes ko fan-out karne deta hai (e.g. list ke har item par alag execution), phir results ko fan-in/reduce kar sakte hain — parallelization ke liye use hota hai.

Q25 (Extra). MCP (Model Context Protocol) kya hai? Ek standardized protocol (Anthropic ka) jo LLM ko external tools/data sources se connect karta hai bina har tool ke liye custom integration likhe — jaise ek common "USB-C port" LLM aur tools ke darmiyan.

C. Scenario-based / Real-World

Q26. Azure DevOps chatbot ~30 tools ke sath ek ReAct agent (create_react_agent) use kar raha hai. Problems aur solution? Bohat saare tools ek hi ReAct agent ko bind karne se LLM ke galat tool choose karne ka chance badhta hai, prompt bloat hota hai (sab tool schemas context mein), aur latency/cost badhti hai. Solution: multi-agent/supervisor pattern — tools ko specialized sub-agents mein split karen (work-item agent, reporting agent), aur supervisor/router node se sahi specialist ko delegate karen.

Q27. Agent ko autonomously refund karna hai, lekin ek threshold se upar human approval chahiye. LangGraph mein kaise design karen? Human-in-the-loop pattern use karen: `analyze` node jo approval-need decide kare, `human_approval` node jahan graph interrupt ho (checkpoint + `thread_id` ke sath resumable), aur `approve_router` conditional edge jo human ke yes/no ke basis par execute/reject route kare.

Q28. 200-300 page annual reports par RAG table/numeric data points miss kar raha hai. Issue aur fix? Poore report ko generic text ki tarah chunk karne se tables ka structure lost ho jata hai, isliye semantically-similar-lekin-galat chunks retrieve hote hain. Fix: tables ko alag extract aur embed karen (ya hybrid retrieval + metadata filtering use karen taake search space narrow ho sahi section tak), sirf plain top-k similarity search par rely na karen.

Q29. Deterministic (supervisor-routed) workflow kab choose karenge fully dynamic ReAct agent ke bajaye? Jab predictability, auditability, ya tight cost/latency control chahiye ho ek well-understood task ke liye jisme kam known branches hon (e.g. rag/llm/web routing classified intent ke basis par). ReAct-style dynamic tool-calling tab behtar hai jab task space open-ended ho aur har path enumerate na ki ja sake — trade-off kam predictability ka.

Q30. Single (mono) agent aur multi-agent architecture mein kaise decide karenge? Simple, narrow tasks ke liye single agent se start karen. Multi-agent/supervisor architecture ki taraf move karen jab task naturally distinct specialties mein decompose ho (research vs finance vs risk vs writing), tool count itna badh

jaye ke ek agent reliably reason na kar sake, ya long-running/deep-agent style planning chahiye ho jahan sub-agents independent chunks handle karein.

Q31 (Extra). Agentic AI aur Generative AI mein basic farq kya hai? Generative AI sirf content generate karta hai (text, image) — ek static request-response. Agentic AI ismein autonomy add karta hai — agent tools use kar sakta hai, multi-step decide kar sakta hai, aur environment ke sath interact kar ke goal achieve kar sakta hai bina har step par human input ke.

Quick Revision Checklist (Interview se Pehle)

- ☐ Assistant → RAG Assistant → AI Agent → Agentic AI ka progression samjha
- ☐ Think-Act-Observe loop aur ReAct pattern yaad hai
- ☐ Deterministic vs Dynamic flow ka farq clear hai
- ☐ Multi-Agent aur Deep Agent ka concept aur Tesla example yaad hai
- ☐ LangGraph ke building blocks (State, Node, Edge, Checkpointer) yaad hain
- ☐ ReAct agent ka graph structure diagram bana sakte hain
- ☐ Pydantic schema ka use tool engineering mein samjha
- ☐ Memory (summary + sliding window) ka flow yaad hai
- ☐ HITL implementation (checkpointer, thread_id, approve_router) yaad hai
- ☐ Guardrails (iteration limit, timeout, stop condition, budget) yaad hain
- ☐ Extra concepts (Agentic Design Patterns, Agentic RAG, MCP/A2A, Vector DB) bhi cover kar liye
- ☐ Scenario-based questions ka practice kar liya